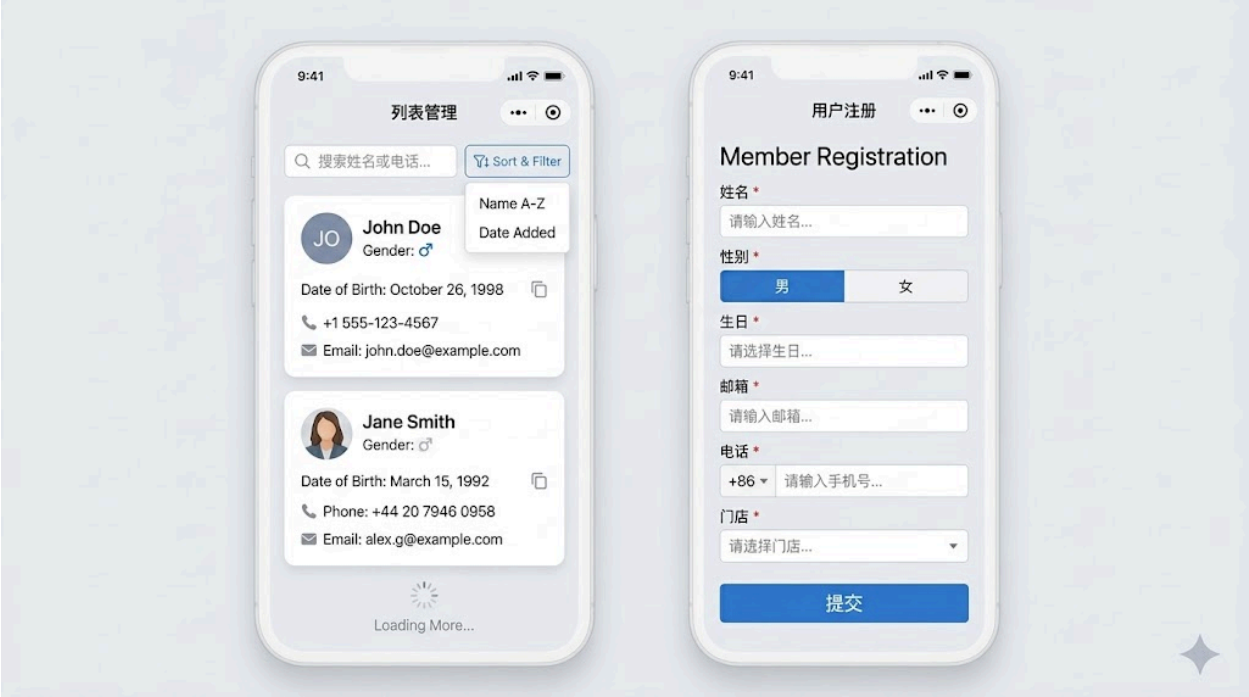
Coding (1~2 hours)



Product Overview

Build a comprehensive User Management System UI for a WeChat Mini Program, explicitly recommended to be developed using the Taro framework to leverage its cross-platform capabilities, modern component design patterns, and state management best practices.
The system should implement a complete CRUD (Create, Read, ~~Update, Delete~~) interface tailored for a Mini Program environment, showcasing Taro component modularity, responsive mobile design, form validation, and Mini Program API integration standards.
Target: Evaluate candidate's ability to implement scalable WeChat Mini Program architecture using Taro with proper separation of concerns, state management, and mobile UI practices.

Our WeChat Mini Program user management interface consists of the following main views and components:

1. **User Registration Form**: User registration and profile creation interface with client-side validation.
2. **User Directory & Search**: Paginated user grid/list display with filtering and sorting capabilities.
3. **User Profile Card/Modal**: Detailed view of individual user information.

Architecture Components

The application must implement the following frontend architectural components:

- **Component Layer**: Modular, reusable Taro UI components for WeChat Mini Program
- **State Management**: Global/local state management (React Context, Redux, or Zustand within Taro)
- **API Service Layer**: Mini Program HTTP request abstraction (Taro.request) for backend API integration
- **Form Handling & Validation**: Schema validation (e.g., React Hook Form, Zod/Yup, or Taro Form components)
- **Styling Architecture**: CSS Modules, Styled Components, or Tailwind CSS
- **Routing & Views**: Taro multi-page routing and navigation system (Taro.navigateTo / switchTab)

Core Requirements (Must Have)

- ✅ Complete CRUD UI workflow implementation in WeChat Mini Program
- ✅ Modular Taro component structure
- ✅ Client-side form validation and dynamic errors
- ✅ Responsive UI design matching provided mocks
- ✅ Data pagination, searching, and sorting
- ✅ Integration with mock API / local server
- ✅ User-friendly error handling and loading states

Bonus Points (Nice to Have)

- 🎯 TypeScript integration with strict type definitions
- 🎯 Comprehensive unit & component testing (React Testing Library / Jest adapted for Taro)
- 🎯 E2E testing (Cypress or Playwright)
- 🎯 Dark mode support or theme customization
- 🎯 Rate limiting and security headers
- 🎯 API versioning strategy

Required Submissions

- Complete Taro project source code
- README.md with setup and running instructions
- Component documentation or Storybook (if applicable)
- Mock API server setup (e.g., json-server, MSW)

All of these should be placed in your personal GitHub repository and shared with reggie.cai@vo2-group.com for review.

Optional Submissions

- Basic test suite with passing tests
- API test scripts
- Security implementation notes

Example Response:

Note that this is just a reference of the API structure that should be included. You can add other data if you think it is necessary.